Structured Logging

each log line is data, not prose

Watch the 4-min video

In one line: write every log line as machine-readable fields (usually JSON) with a level and automatic context such as request, tenant and trace IDs, so logs can be filtered, counted and joined across services; and never let secrets or sensitive personal data into them.

1 Regex archaeology at 2 a.m.

An incident is running. You need every error for one customer in the last hour. The logs say `user did something, failed, retrying`, each team in its own format. You write regular expressions that miss half the lines. Free text is written for a human reading one line; during an incident you need a machine to search millions.

2 Each log line is data

Free text	Structured (JSON)
`Order 991 failed for acme after 2300ms`	`{"level":"error","event":"order failed","order_id":991, "tenant_id":"acme","duration_ms":2300,"route":"/checkout"}`
search = regex, breaks when wording changes	search = query: `tenant_id=acme AND level=error`

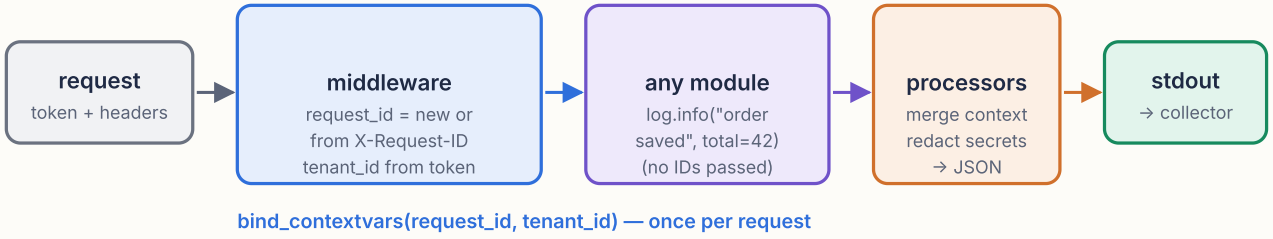
A good line has a **timestamp** (UTC, ISO 8601), a **level**, a short constant **event** name, and named fields. Keep the event name constant (`"order failed"`) and put the variable parts in fields, so identical events group together.

3 Levels, with discipline

Level	Means	Example	In production
DEBUG	developer detail	cache key computed	off; switch on per service when needed
INFO	normal, important event	request completed, job finished	on, often sampled for noisy paths
WARNING	unexpected, but coped	retry succeeded, fallback used	on
ERROR	a request or job failed	payment call failed after retries	on, always kept

If everything is ERROR, the level means nothing and alerts built on it become noise. A 404 from a user typo is not an error of *your* service.

4 Context on every line, automatically



Result on every line, from every module:

```
{"level": "info", "event": "order saved", "total": 42, "request_id": "r-81c", "tenant_id": "acme"}
```

Figure 1. Middleware binds the IDs once; every log call in that request, in any module, inherits them. Processors add level and time, drop secrets and render JSON.

Add **request ID, trace ID, tenant ID, user ID, service name and version** to every line using middleware and context variables, never by hand in each call. In Python, **structlog** (contextvars) or the standard `logging` module with a JSON formatter and a filter both work. With OpenTelemetry, the trace and span IDs can be injected too, so you can jump from a trace to its logs (lesson 65). Write to stdout and let the platform ship the stream (the Twelve-Factor App rule; lesson 66).

5 What never to log

Passwords, session tokens, API keys, JWTs, full card numbers, and raw request bodies that may contain them; also sensitive personal data beyond what you need (national IDs, health data). Logs are copied to many systems, kept for months and read by many people (OWASP Logging Cheat Sheet). Enforce it in the pipeline: a processor or filter that drops or masks known keys (`password`, `authorization`, `card`), so one careless line can't leak them. Prefer allow-lists of fields over logging whole objects.

6 Volume and cost

- **Sample** low-value logs (successful health checks, 200s on hot paths); keep **every** error.
- **Retention per level:** e.g. errors 30–90 days, debug a few days.
- **One rich line per request** with all context usually beats twenty thin lines.
- **Runtime-switchable** debug logging, so you can turn it on for one service during an incident without a deploy.

```
structlog.configure(processors=[merge_contextvars, add_log_level,
    TimeStamper(fmt="iso"), drop_secrets, JSONRenderer()])   # drop_secrets: yours

@app.middleware("http")
async def log_context(request, call_next):
    clear_contextvars()
    bind_contextvars(request_id=request.headers.get("x-request-id") or uuid4().hex,
                     tenant_id=tenant_from_verified_token(request))
    return await call_next(request)
```

Figure 2. The video's setup (structlog, names imported from structlog.contextvars and structlog.processors): JSON output plus FastAPI middleware that binds request and tenant IDs.

WORKED EXAMPLE: SUPPORT TICKET FROM TENANT ACME

The FastAPI middleware above generates (or accepts) a request ID and reads the tenant from the **verified** token, not from a client header the caller could forge. Every line in that request now carries both. Support receives "checkout failed around 14:00" from acme and runs `tenant_id=acme AND level=error` for 13:45–14:15. In seconds they see the failed requests, each with its `request_id`; searching one of those IDs pulls up that request's whole story across modules: a query, not a regex hunt.

COMMON MISTAKES

- Logging the whole request or user object "for debugging" and leaking tokens or personal data.
- Variable data in the message string, so identical events can't be grouped.
- Passing request IDs by hand through every function: someone forgets, and lines go missing from the story.
- Logging expected client errors (404, validation) at ERROR, drowning the real failures.

7 Key terms

Structured log: log line as named fields, usually JSON

Log level: severity: DEBUG, INFO, WARNING, ERROR

Context variables: per-request values merged into every line

Request ID: unique ID for one request, on every line

Redaction: dropping or masking sensitive fields in the pipeline

Sampling: keeping only a fraction of low-value logs

8 Interview questions

Q1. Why structured logs? BASIC

Machine-readable fields make logs searchable, filterable and joinable across services, especially with request and trace IDs on every line.

Q2. What should never be logged? BASIC

Passwords, tokens, API keys, full card numbers and other sensitive personal data; mask or drop them in the logging pipeline, not by convention alone.

Q3. How do you get the request ID onto every log line without changing every function? SCENARIO

Bind it once in middleware into context variables (structlog contextvars, or a logging filter reading a ContextVar); the formatter merges it into each line.

Q4. Your logging bill doubled. What do you cut first? SCENARIO

Sample successful high-volume lines like health checks, merge many thin lines into one per request, shorten retention for low levels, but keep all errors.

Q5. Should the tenant ID in logs come from the X-Tenant header? TRICKY

Not if a client controls it: take it from the verified token or session, or an attacker can make their activity appear under another tenant.

Q6. Is a 404 an ERROR? TRICKY

Usually not for your service: it's a correct response to a bad request. Log it at INFO or WARNING and keep ERROR for failures you must fix.

Next: SD-N-003 · Metrics Types and Design backendroadmap.com